10 – Hands-on Practice Exercises

The goal of this chapter is to give you a **sandbox** where you can tinker with the real codebase, reinforce the concepts you learned in the previous chapters, and build confidence to make small modifications.

Each exercise follows the same template:

1. **Target file** – absolute path of the file you will edit.
2. **Task description** – what you need to change or add.
3. **Hints** – helpful pointers, functions you may need to touch, and where to look for related code.
4. **Solution** – a complete implementation that you can compare against after you finish. (Hidden by default – click **Show Solution**.)

Exercise 1 – Change the Flat Shipping Fee

Target file: `e:/intenship/easyshop/includes/functions.php` **Task:** Replace the hard-coded shipping fee of **Rs. 200** with a configurable constant `$FLAT_SHIPPING_FEE`. The value should be easy to modify in one place.

Hints:

- Locate the function `calculate_shipping()` (around lines 430-442). It currently returns the literal `200`.
- Define a global constant at the top of the file, e.g., `define('FLAT_SHIPPING_FEE', 200);`.
- Use the constant inside the function instead of the literal.

▶ Show Solution

Exercise 2 – Add a New Column to the `products` Table

Target file: `e:/intenship/easyshop/database/easyshop_production.sql` **Task:** Add a `brand VARCHAR(100) NULL` column to `products`. Then update the admin **Add Product** form to accept a brand name and store it.

Hints:

- In the SQL file, find the `CREATE TABLE products` statement and add the column after `price`.
- In `admin/products/add.php` you will see the HTML form – add a new `<input>` for `brand`.
- In `admin/products/save.php` (or the script that processes the form) add the `brand` field to the prepared statement.

▶ Show Solution

Exercise 3 – Change Product Sorting on the Storefront

Target file: `e:/intenship/easyshop/index.php` **Task:** By default products are displayed **newest first** (`ORDER BY created_at DESC`). Change the default to **price ascending**.

Hints:

- Look for the SQL query that fetches products for the home page (around line 120).
- Replace `created_at DESC` with `price ASC`.
- Test the change by reloading the home page – cheaper items should appear at the top.

▶ Show Solution

Exercise 4 – Add a New Dashboard Metric (Total Revenue)

Target file: `e:/intenship/easyshop/admin/index.php` **Task:** Show the **total revenue** (sum of `order_items.subtotal`) for all **delivered** orders in the admin dashboard.

Hints:

- Inside `admin/index.php` you will find three aggregate queries (product, category, customer). Add a fourth one.
- Use `SELECT IFNULL(SUM(oi.subtotal),0) AS total_revenue FROM orders o JOIN order_items oi ON o.id = oi.order_id WHERE o.order_status = 'delivered';`
- Echo the value inside a new Bootstrap card similar to the existing ones.

▶ Show Solution

Exercise 5 – Strengthen Email Validation on Registration

Target file: `e:/intenship/easyshop/actions/register-action.php` **Task:** Replace the simple `filter_var($email, FILTER_VALIDATE_EMAIL)` check with a **regex** that enforces a Pakistani domain (e.g., `.pk`) *or* any public domain but disallows common disposable email providers.

Hints:

- The current validation is on line 55.
- Use a regex like `/^[A-Za-z0-9._%+-]+@(?!mailinator\.com|trashmail\.com)[A-Za-z0-9.-]+\.(pk|com|org|net)$/i`.
- If the regex fails, set an error flash message similar to the existing flow.

▶ Show Solution

Exercise 6 – Add a Simple Rate-Limiter on Login Attempts

Target file: `e:/intenship/easyshop/actions/login-action.php` **Task:** Implement a **basic per-IP rate limiter** that blocks further login attempts after **5 failed attempts** within **15 minutes**.

Hints:

- Create a new table `login_attempts` (IP, attempts, last_attempt DATETIME) – you can add it to the `database/easyshop_production.sql`.
- In `login-action.php`, after a failed password check, increment the counter for `$_SERVER['REMOTE_ADDR']`.
- Before processing a login, query the table: if `attempts >= 5` and `last_attempt` within 15 minutes, abort with an error flash.
- On successful login, delete the record for that IP.

▶ Show Solution

Exercise 7 – Create a Simple Search Endpoint (API style)

Target file: `e:/intenship/easyshop/api/search.php` **Task:** Build a lightweight **JSON API** that accepts a query parameter `q` and returns matching products (name, price, thumbnail URL). No front-end UI needed – only the endpoint.

Hints:

- Use `header('Content-Type: application/json');`.
- Perform a `SELECT id, name, price, image FROM products WHERE name LIKE ? LIMIT 10;` with `%$q%` bound.
- Encode the result with `json_encode`.
- Place the file under a new folder `api/` (create it). Add a route link in `README.md` for developers.

▶ Show Solution

How to Use These Exercises

1. **Back up** the repository or work on a separate Git branch (`git checkout -b practice`).
2. Follow the tasks **in order** – each builds on concepts from earlier chapters.
3. After completing an exercise, run the relevant part of the application locally (e.g., `php -S localhost:8000`) and verify the change.
4. Compare your solution with the hidden one. If you diverge but it works, keep the improvement and note it in a personal log.
5. Commit each exercise with a clear message, e.g., `git commit -m "Exercise 1 – configurable shipping fee"`.

Enjoy hacking the code!

Prepared by the senior core-PHP developer (AI-assisted).